

() COMPILER DESIGN

UNIT II

Syntax analysis: the role of the parser, context free grammars, writing a grammar: eliminating ambiguity, elimination of left recursion, elimination of left factoring

Top-down parsing: Recursive descent parsing, first and follow, predictive parsing, construction of predictive parsing table.

Syntax Analysis:

By design, every programming language has precise rules that prescribe the syntactic structure of well-formed programs. In C, for example, a program is made up of functions, a function out of declarations and statements, a statement out of expressions, and so on. The syntax of programming language constructs can be specified by context-free grammars or BNF (Backus-Naur Form) notation.

Grammars offer significant benefits for both language designers and compiler writers.

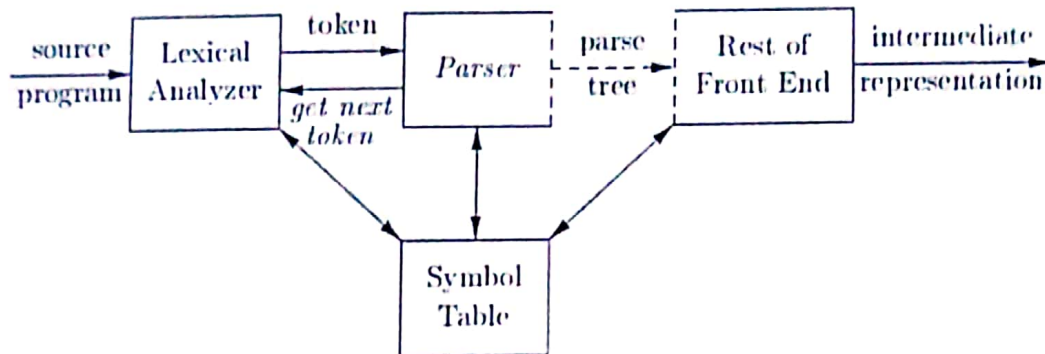
- A grammar gives a precise, yet easy-to-understand, syntactic specification of a programming language.
- From certain classes of grammars, we can construct automatically an efficient parser that determines the syntactic structure of a source program. As a side benefit, the parser-construction process can reveal syntactic ambiguities and trouble spots that might have slipped through the initial design phase of a language.
- The structure imparted to a language by a properly designed grammar is useful for translating source programs into correct object code and for detecting errors.
- A grammar allows a language to be evolved or developed iteratively, by adding new constructs to perform new tasks. These new constructs can be integrated more easily into an implementation that follows the grammatical structure of the language.

The Role of Parser:

In our compiler model, the parser obtains a string of tokens from the lexical analyzer, as shown in Fig. below, and verifies that the string of token names can be generated by the grammar for the source language. We expect the parser to report any syntax errors in an intelligible fashion and to recover from commonly occurring errors to continue processing the remainder of the program.

COMPILER DESIGN

Conceptually, for well-formed programs, the parser constructs a parse tree and passes it to the rest of the compiler for further processing. In fact, the parse tree need not be constructed explicitly, since checking and translation actions can be interspersed with parsing, as we shall see. Thus, the parser and the rest of the front end could well be implemented by a single module.



There are three general types of parsers for grammars: universal, top-down, and bottom-up.

The methods commonly used in compilers can be classified as being either top-down or bottom-up. As implied by their names, top-down methods build parse trees from the top (root) to the bottom (leaves), while bottom-up methods start from the leaves and work their way up to the root. In either case, the input to the parser is scanned from left to right, one symbol at a time.

Context-free Grammars:

A context-free grammar (grammar for short) consists of terminals, nonterminals, a start symbol, and productions.

1. Terminals are the basic symbols from which strings are formed. The term "token name" is a synonym for "terminal" and frequently we will use the word "token" for terminal when it is clear that we are talking about just the token name. We assume that the terminals are the first components of the tokens output by the lexical analyzer.

Consider the production

$$\text{stmt} \rightarrow \text{if (expr) stmt else stmt}$$

In the above production, the terminals are the keywords **if** and **else** and the symbols "(" and ")".

2. Nonterminals are syntactic variables that denote sets of strings. In the above production, **stmt** and **expr** are nonterminals. The sets of strings denoted by nonterminals help define the language generated by the grammar. Nonterminals impose a hierarchical structure on the language that is key to syntax analysis and translation.
3. In a grammar, one nonterminal is distinguished as the start symbol, and the set of strings it denotes is the language generated by the grammar. Conventionally, the productions for the start symbol are listed first.
4. The productions of a grammar specify the manner in which the terminals and nonterminals can be combined to form strings. Each production consists of:
 - a) A nonterminal called the head or left side of the production; this production defines some of the strings denoted by the head.
 - b) The symbol $\rightarrow$. Sometimes $::=$ has been used in place of the arrow.
 - c) A body or right side consisting of zero or more terminals and nonterminals.

Grammar for simple arithmetic expressions

expression $\rightarrow$ expression + term

expression $\rightarrow$ expression - term

expression $\rightarrow$ term

term $\rightarrow$ term * factor

term $\rightarrow$ term / factor

term $\rightarrow$ factor

factor $\rightarrow$ (expression)

factor $\rightarrow$ id

The grammar shown above defines simple arithmetic expressions. In this grammar, the terminal symbols are id + - * / (). The nonterminal symbols are expression, term and factor, and expression is the start symbol.

Notational Conventions:

To avoid always having to state that "these are the terminals," "these are the nonterminals," and so on, the following notational conventions for grammars will be used.

1. These symbols are terminals:

- (a) Lowercase letters early in the alphabet, such as a, b, c.
- (b) Operator symbols such as +, -, and so on.
- (c) Punctuation symbols such as parentheses, comma, and so on.
- (d) The digits 0, 1, ..., 9.
- (e) Boldface strings such as **id** or **if**, each of which represents a single terminal symbol.

2. These symbols are nonterminals:

- (a) Uppercase letters early in the alphabet, such as A, B, C.
- (b) The letter S, which, when it appears, is usually the start symbol.
- (c) Lowercase, italic names such as *expr* or *stmt*.
- (d) When discussing programming constructs, uppercase letters may be used to represent nonterminals for the constructs. For example, nonterminals for expressions, terms, and factors are often represented by E, T, and F, respectively.

3. Uppercase letters late in the alphabet, such as X, Y, Z, represent grammar symbols; that is, either nonterminals or terminals.

4. Lowercase letters late in the alphabet, u, v, ..., z, represent (possibly empty) strings of terminals.

5. Lowercase Greek letters, α , β , γ for example, represent (possibly empty) strings of grammar symbols. Thus, a generic production can be written as $A \rightarrow \alpha$, where A is the head and α the body.

6. A set of productions $A \rightarrow \alpha_1$, $A \rightarrow \alpha_2$, ..., $A \rightarrow \alpha_k$ with a common head A (call them A-productions), may be written $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_k$. Call α_1 , α_2 , ..., α_k the alternatives for A.

7. Unless stated otherwise, the head of the first production is the start symbol.

Using these conventions, the grammar can be rewritten concisely as

$$\begin{aligned} E &\rightarrow E + T \mid E - T \mid T \\ T &\rightarrow T * F \mid T / F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Derivations:

The construction of a parse tree can be made precise by taking a derivational view, in which productions are treated as rewriting rules. Beginning with the start symbol, each rewriting step replaces a nonterminal by the body of one of its productions. This derivational view corresponds to the top-down construction of a parse tree, but the precision afforded by derivations will be especially helpful when bottom-up parsing is discussed. As we shall see, bottom-up parsing is related to a class of derivations known as "rightmost" derivations, in which the rightmost nonterminal is rewritten at each step.

For example, consider the following grammar, with a single nonterminal E

$$E \rightarrow E + E \mid E * E \mid - E \mid (E) \mid \text{id}$$

The production $E \rightarrow (E)$ can be applied to replace any instance of E in any string of grammar symbols by (E), e.g., $E * E \rightarrow (E) * E$ or $E * E \rightarrow E * (E)$. We can take a single E and repeatedly apply productions in any order to get a sequence of replacements.

For example,

$$E \rightarrow -E \rightarrow -(E) \rightarrow -(id)$$

We call such a sequence of replacements a derivation of -(id) from E. This derivation provides a proof that the string -(id) is one particular instance of an expression.

For a general definition of derivation, consider a nonterminal A in the middle of a sequence of grammar symbols, as in $\alpha A \beta$, where α and β are arbitrary strings of grammar symbols. Suppose $A \rightarrow \gamma$ is a production. Then, we write $\alpha A \beta \rightarrow \alpha \gamma \beta$. The symbol $\rightarrow$ means, "derives in one step." When a sequence of derivation steps $\alpha_1 \rightarrow \alpha_2 \rightarrow \dots \rightarrow \alpha_n$ rewrites α_1 to α_n , we say α_1 derives α_n . Often, we wish to say, "derives in zero or more steps."

we can use the symbol $\Rightarrow^*$. Thus,

1. $\alpha \Rightarrow^* \alpha$, for any string α , and
2. If $\alpha \Rightarrow^* \beta$ and $\beta \Rightarrow \gamma$, then $\alpha \Rightarrow^* \gamma$.

Likewise, $\Rightarrow^+$ means, "derives in one or more steps."

If $S \rightarrow \alpha$, where S is the start symbol of a grammar G , we say that α is a sentential form of G . Note that a sentential form may contain both terminals and nonterminals, and may be empty. A sentence of G is a sentential form with no nonterminals. The language generated by a grammar is its set of sentences.

Thus, a string of terminals w is in $L(G)$, the language generated by G , if and only if w is a sentence of G . A language that can be generated by a grammar is said to be a context-free language. If two grammars generate the same language, the grammars are said to be equivalent.

The string $-(id + id)$ is a sentence of grammar because there is a derivation

$$E \rightarrow -E \rightarrow -(E) \rightarrow -(E+E) \rightarrow -(id+E) \rightarrow -(id+id) \quad \dots(1)$$

The strings $E, -E, -(E), \dots, -(id + id)$ are all sentential forms of this grammar. We write $E \rightarrow -(id + id)$ to indicate that $-(id + id)$ can be derived from E .

At each step in a derivation, there are two choices to be made. We need to choose which nonterminal to replace, and having made this choice, we must pick a production with that nonterminal as head. For example, the following alternative derivation of $-(id+id)$ differs from derivation shown above in the last two steps:

$$E \rightarrow -E \rightarrow -(E) \rightarrow -(E+E) \rightarrow -(E+id) \rightarrow -(id+id) \quad \dots(2)$$

Each nonterminal is replaced by the same body in the two derivations, but the order of replacements is different.

To understand how parsers work, we shall consider derivations in which the nonterminal to be replaced at each step is chosen as follows:

1. In leftmost derivations, the leftmost nonterminal in each sentential is always chosen. If $\alpha \rightarrow \beta$ is a step in which the leftmost nonterminal in α is replaced, we write $\alpha \rightarrow_{lm} \beta$

2. In rightmost derivations, the rightmost nonterminal is always chosen; we write $\alpha \rightarrow_{rm} \beta$ in this case.

Derivation (1) is leftmost, so it can be rewritten as

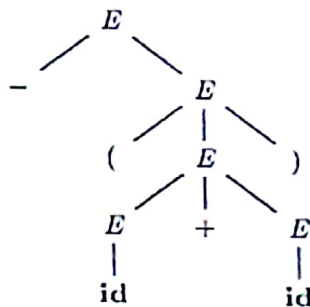
$E \rightarrow_{lm} -E \rightarrow_{lm} -(E) \rightarrow_{lm} -(E+E) \rightarrow_{lm} -(id+E) \rightarrow_{lm} -(id+id)$

Note that (2) is a rightmost derivation.

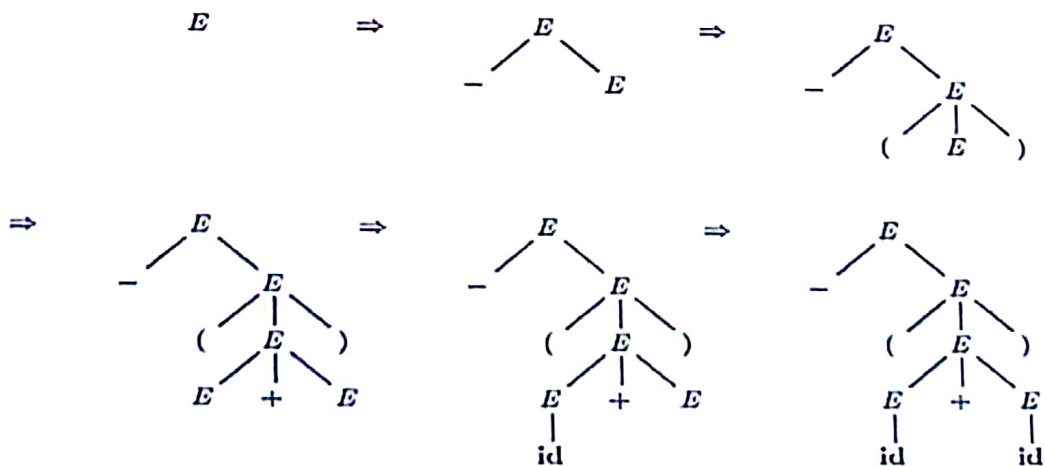
Parse Trees and Derivations:

A parse tree is a graphical representation of a derivation that filters out the order in which productions are applied to replace nonterminals. Each interior node of a parse tree represents the application of a production. The interior node is labeled with the nonterminal A in the head of the production; the children of the node are labeled, from left to right, by the symbols in the body of the production by which this A was replaced during the derivation.

For example, the parse tree for $-(id + id)$ in Fig. below.



Sequence of parse trees for derivation (1) is shown below:



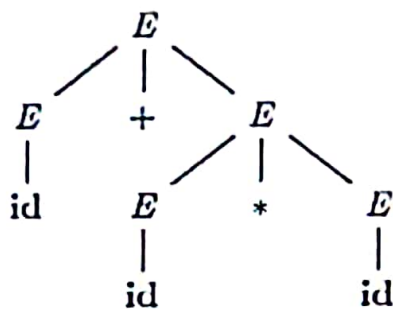
Ambiguity:

A grammar that produces more than one parse tree for some sentence is said to be ambiguous. Put another way, an ambiguous grammar is one that produces more than one leftmost derivation or more than one rightmost derivation for the same sentence.

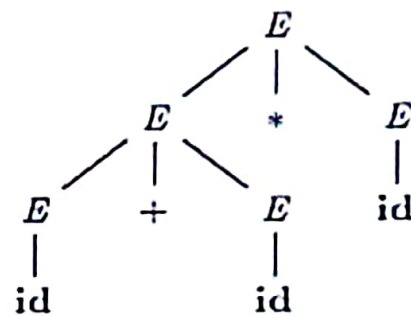
Eg: The arithmetic expression grammar permits two distinct leftmost derivations for the sentence $\text{id} + \text{id} * \text{id}$:

$$\begin{array}{ll}
 E \Rightarrow E + E & E \Rightarrow E * E \\
 \Rightarrow \text{id} + E & \Rightarrow E + E * E \\
 \Rightarrow \text{id} + E * E & \Rightarrow \text{id} + E * E \\
 \Rightarrow \text{id} + \text{id} * E & \Rightarrow \text{id} + \text{id} * E \\
 \Rightarrow \text{id} + \text{id} * \text{id} & \Rightarrow \text{id} + \text{id} * \text{id}
 \end{array}$$

The corresponding parse trees are shown below:



(a)



(b)

Note that the parse tree of Fig. (a) reflects the commonly assumed precedence of + and *, while the tree of Fig. (b) does not. That is, it is customary to treat operator * as having higher precedence than +, corresponding to the fact that we would normally evaluate an expression like $a + b * c$ as $a + (b * c)$, rather than as $(a + b) * c$.

For most parsers, it is desirable that the grammar be made unambiguous, for if it is not, we cannot uniquely determine which parse tree to select for a sentence. In other cases, it is convenient to use carefully chosen ambiguous grammars, together with disambiguating rules that "throw away" undesirable parse trees, leaving only one tree for each sentence.

Consider the context-free grammar:

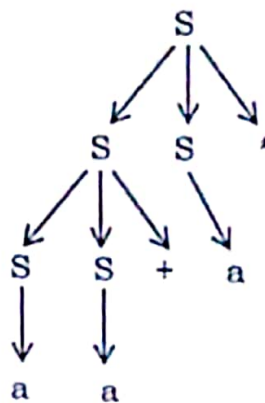
$$S \rightarrow SS+ \mid SS^* \mid a$$

and the string $aa + a^*$.

- Give a leftmost derivation for the string.
- Give a rightmost derivation for the string.
- Give a parse tree for the string.

Left most derivation : $S \rightarrow SS^* \rightarrow SS+S^* \rightarrow aS+S^* \rightarrow aa+S^* \rightarrow aa+a^*$

Right most derivation : $S \rightarrow SS^* \rightarrow Sa^* \rightarrow SS+a^* \rightarrow Sa+a^* \rightarrow aa+a^*$



Useless symbol:

A grammar symbol X (terminal or nonterminal) is useless if there is no derivation of the form $S \rightarrow wXy \rightarrow wxy$. That is, X can never appear in the derivation of any sentence.

- Give an algorithm to eliminate from a grammar all productions containing useless symbols.
- Apply your algorithm to the grammar:

$$S \rightarrow 0 \mid A$$

$$A \rightarrow AB$$

$$B \rightarrow 1$$

A symbol X is useful if:

- If X is generating, i.e., $X \Rightarrow^* w$, where $w \in L(G)$ and w in V_t^* , this means that the string leads to a string of terminal symbols.
- If X is reachable If there is a derivation $S \Rightarrow^* \alpha X \beta \Rightarrow^* w$, $w \in L(G)$, for same α and β , then X is said to be reachable.

A number that is useful is both generating and reachable. For reduction of a given grammar G:

1. Identify non-generating symbols in the given CFG and eliminate those productions which contains non-generating symbols.
2. Identify non-reachable symbols and eliminate those productions which contain the non-reachable symbols

Example: Remove the useless symbol from the given context free grammar:

$S \rightarrow aB / bX$
 $A \rightarrow Ba d / bSX / a$
 $B \rightarrow aSB / bBX$
 $X \rightarrow SBD / aBx / ad$

Solution:

A and X directly derive string of terminals a and ad, hence they are useful. Since X is a useful symbol so S is also a useful symbol as $S \rightarrow bX$. But B does not derive any string w in V^*t so clearly B is a non-generating symbol. So eliminating those productions with B in them we get

$S \rightarrow bX$
 $A \rightarrow bSX / a$
 $X \rightarrow ad$

In the reduced grammar A is a non-reachable symbol so we remove it and the final grammar after elimination of the useless symbols is

$S \rightarrow bX$
 $X \rightarrow ad$

In the following grammar

$S \rightarrow 0 \mid A$
 $A \rightarrow AB$
 $B \rightarrow 1$

A is not generating any string of terminals. So $A \rightarrow AB$ can be removed, in turn we can remove $S \rightarrow A$. In the reduced grammar B is a non-reachable symbol so we remove it.

Final grammar is $S \rightarrow 0$.

Eliminating Ambiguity:

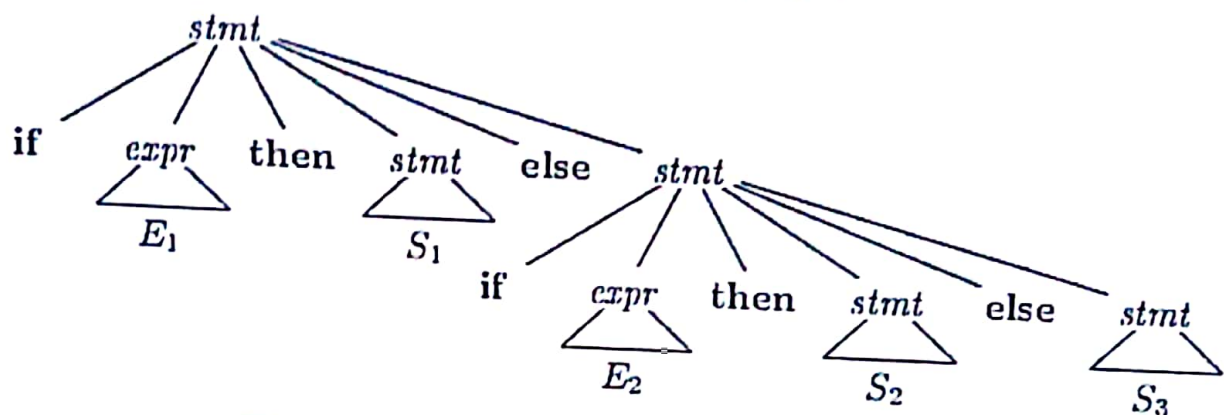
Sometimes an ambiguous grammar can be rewritten to eliminate the ambiguity. As an example, we shall eliminate the ambiguity from the following "dangling- else" grammar:

```

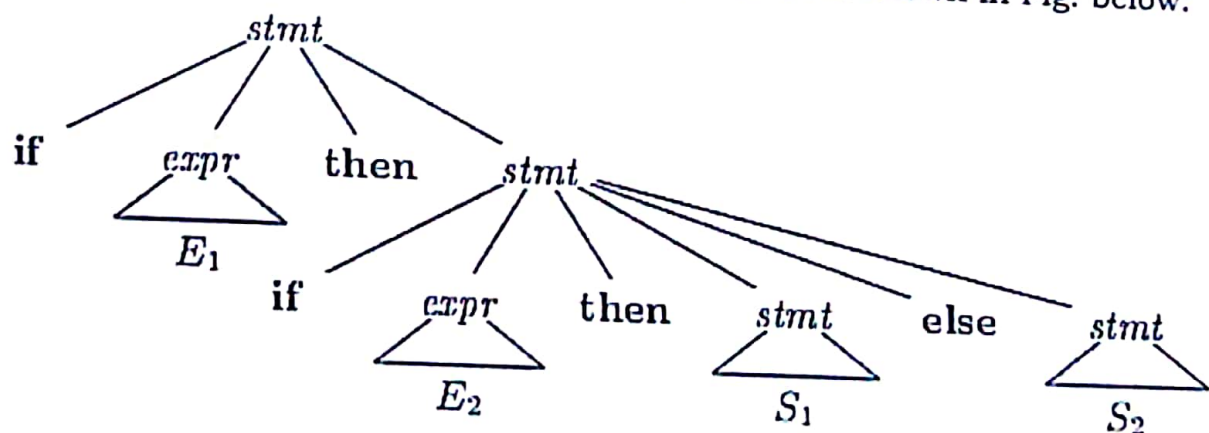
stmt  →  if expr then stmt
        |  if expr then stmt else stmt
        |  other
    
```

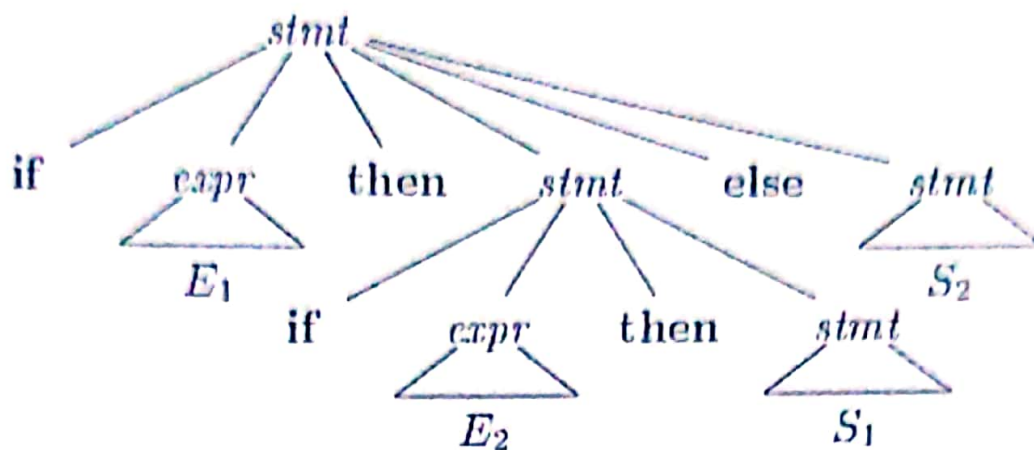
Here "other" stands for any other statement. According to this grammar, the compound conditional statement

if E1 then S1 else if E2 then S2 else S3 has the parse tree shown in Fig. below:



Grammar shown above is ambiguous since the string if E1 then if E2 then S1 else S2 has the two parse trees shown in Fig. below:





In all programming languages with conditional statements of this form, the first parse tree is preferred. The general rule is, "Match each else with the closest unmatched then."

We can rewrite the dangling-else grammar as the following unambiguous grammar. The idea is that a statement appearing between a then and an else must be "matched"; that is, the interior statement must not end with an unmatched or open then. A matched statement is either an if-then-else statement containing no open statements or it is any other kind of unconditional statement. Thus, we may use the grammar in Fig. below:

<i>stmt</i>	→	<i>matched_stmt</i>
		<i>open_stmt</i>
<i>matched_stmt</i>	→	if <i>expr</i> then <i>matched_stmt</i> else <i>matched_stmt</i>
		other
<i>open_stmt</i>	→	if <i>expr</i> then <i>stmt</i>
		if <i>expr</i> then <i>matched_stmt</i> else <i>open_stmt</i>

This grammar generates the same strings as the dangling-else grammar, but it allows only one parsing for string; namely, the one that associates each else with the closest previous unmatched then.

Elimination of left recursion:

A grammar is left recursive if it has a nonterminal A such that there is a derivation $A \rightarrow A\alpha$ for some string α . Top-down parsing methods cannot handle left-recursive grammars, so a transformation is needed to eliminate left recursion.

$A \rightarrow A\alpha \mid \beta$ can be replaced by

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' \mid \epsilon \end{aligned}$$

Consider the grammar

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid id \end{aligned}$$

After eliminating left recursion

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \epsilon \\ F &\rightarrow (E) \mid id \end{aligned}$$

Remove the left recursion from $A \rightarrow AX \mid Y$

$$A \rightarrow YA'$$

$$A' \rightarrow XA' \mid \epsilon$$

Left Factoring:

Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive, or top-down, parsing. When the choice between two alternative A-productions is not clear, we may be able to rewrite the productions to defer the decision until enough of the input has been seen that we can make the right choice.

For example, if we have the two productions

$$\text{stmt} \rightarrow \text{if expr then stmt else stmt} \mid \text{if expr then stmt}$$

On seeing the input if, we cannot immediately tell which production to choose to expand stmt. In general, if $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ are two A-productions, and the input begins with a nonempty string derived from α , we do not know whether to expand A to $\alpha\beta_1$ or $\alpha\beta_2$. However, we may defer the decision by expanding A to $\alpha A'$. Then, after seeing the input derived from α , we expand A' to β_1 or to β_2 . That is, left-factored, the original productions become

$$\begin{aligned} A &\rightarrow \alpha A' \\ A' &\rightarrow \beta_1 \mid \beta_2 \end{aligned}$$

Eg1: Do left factoring for $A \rightarrow XB \mid XC$

$$\begin{aligned} A &\rightarrow XA' \\ A' &\rightarrow B \mid C \end{aligned}$$

Eg2:

$$\begin{aligned} S &\rightarrow iEtS \mid iEtSeS \mid a \\ E &\rightarrow b \end{aligned}$$

After left factoring

$$\begin{aligned} S &\rightarrow iEtSS' \mid a \\ S' &\rightarrow eS \mid \epsilon \\ E &\rightarrow b \end{aligned}$$

Eg3:

$$S \rightarrow abcde \mid abcf \mid abg \mid ae$$

After left factoring

$$\begin{aligned} S &\rightarrow aS' \\ S' &\rightarrow bB' \mid e \\ B' &\rightarrow cC' \mid g \\ C' &\rightarrow de \mid f \end{aligned}$$

Eg4:

$$S \rightarrow ab \mid ac \mid ad \mid ae$$

After left factoring

$$\begin{aligned} S &\rightarrow aS' \\ S' &\rightarrow b \mid c \mid d \mid e \end{aligned}$$

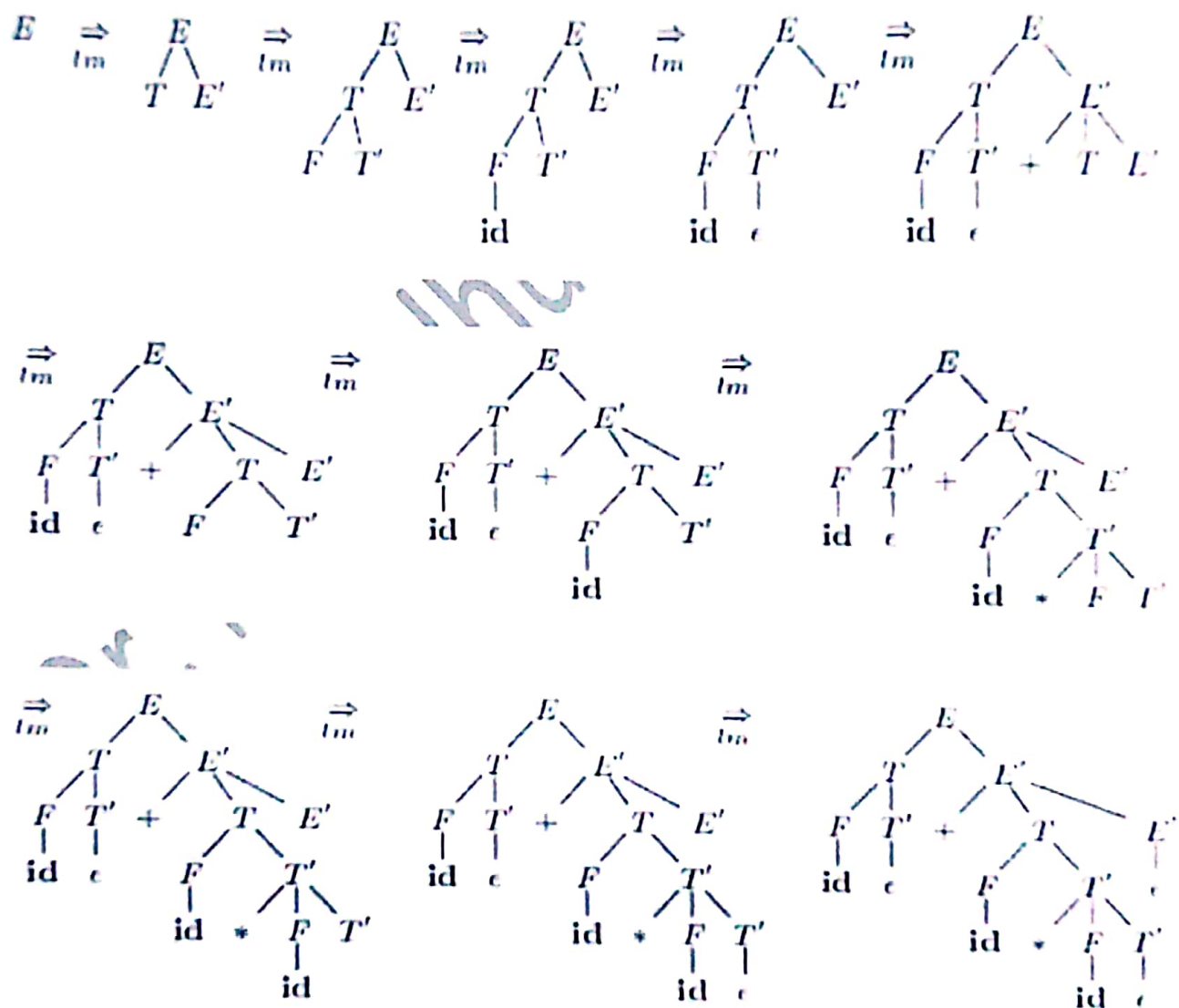
Top-down parsing:

Top-down parsing can be viewed as the problem of constructing a parse tree for the input string, starting from the root and creating the nodes of the parse tree in preorder. Equivalently, top-down parsing can be viewed as finding a leftmost derivation for an input string.

The sequence of parse trees shown in Fig. below, for the input $\text{id}+\text{id}*\text{id}$ is a top-down parse according to grammar:

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \epsilon \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \epsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

This sequence of trees corresponds to a leftmost derivation of the input.



At each step of a top-down parse, the key problem is that of determining the production to be applied for a nonterminal, say A. Once an A-production is chosen, the rest of the parsing process consists of "matching" the terminal symbols in the production body with the input string.

Recursive-descent parsing:

A recursive-descent parsing program consists of a set of procedures, one for each nonterminal. Execution begins with the procedure for the start symbol, which halts and announces success if its procedure body scans the entire input string. Pseudocode for a typical nonterminal appears below:

```
void A() {  
1)      Choose an A-production,  $A \rightarrow X_1 X_2 \cdots X_k$ ;  
2)      for (  $i = 1$  to  $k$  ) {  
3)          if (  $X_i$  is a nonterminal )  
4)              call procedure  $X_i()$ ;  
5)          else if (  $X_i$  equals the current input symbol  $a$  )  
6)              advance the input to the next symbol;  
7)          else /* an error has occurred */;  
      }  
}
```

FIRST and FOLLOW:

The construction of both top-down and bottom-up parsers is aided by two functions, FIRST and FOLLOW, associated with a grammar G. During top-down parsing, FIRST and FOLLOW allow us to choose which production to apply, based on the next input symbol.

Define $FIRST(a)$, where a is any string of grammar symbols, to be the set of terminals that begin strings derived from a .

Define $FOLLOW(A)$, for nonterminal A, to be the set of terminals a that can appear immediately to the right of A in some sentential form; that is, the set of terminals a such that there exists a derivation of the form $S \rightarrow \alpha A a \beta$, for some α and β .

To compute $FIRST(X)$ for all grammar symbols X, apply the following rules until no more terminals or ϵ can be added to any FIRST set.

1. If X is a terminal, then $FIRST(X) = \{X\}$.
2. If X is a nonterminal and $X \rightarrow Y_1 Y_2 \dots Y_k$ is a production for some $k \geq 1$, then place 'a' in $FIRST(X)$ if for some i , a is in $FIRST(Y_i)$, and ϵ is in all of $FIRST(Y_1), \dots, FIRST(Y_{i-1})$; that is, $Y_1 \dots Y_{i-1} \rightarrow \epsilon$. If ϵ is in $FIRST(Y_j)$ for all $j = 1, 2, \dots, k$, then add ϵ to $FIRST(X)$. For example, everything in $FIRST(Y_1)$ is surely in $FIRST(X)$. If Y_1 does not derive ϵ , then we add nothing more to $FIRST(X)$, but if $Y_1 \rightarrow \epsilon$, then we add $FIRST(Y_2)$, and so on.
3. If $X \rightarrow \epsilon$ is a production, then add ϵ to $FIRST(X)$.

To compute $FOLLOW(A)$ for all nonterminals A , apply the following rules until nothing can be added to any $FOLLOW$ set.

1. Place $\$$ in $FOLLOW(S)$, where S is the start symbol, and $\$$ is the input right endmarker.
2. If there is a production $A \rightarrow \alpha B \beta$, then everything in $FIRST(\beta)$ except ϵ is in $FOLLOW(B)$.
3. If there is a production $A \rightarrow \alpha B$, or a production $A \rightarrow \alpha B \beta$, where $FIRST(\beta)$ contains ϵ , then everything in $FOLLOW(A)$ is in $FOLLOW(B)$.

Consider the following grammar

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \epsilon \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \epsilon \\ F &\rightarrow (E) \mid id \end{aligned}$$

1. $FIRST(F) = FIRST(T) = FIRST(E) = \{ (, id \}$. To see why, note that the two productions for F have bodies that start with these two terminal symbols, id and the left parenthesis. T has only one production, and its body starts with F . Since F does not derive ϵ , $FIRST(T)$ must be the same as $FIRST(F)$. The same argument covers $FIRST(E)$.
2. $FIRST(E') = \{ +, \epsilon \}$. The reason is that one of the two productions for E' has a body that begins with terminal $+$, and the other's body is ϵ . Whenever a nonterminal derives ϵ , we place ϵ in $FIRST$ for that nonterminal.
3. $FIRST(T') = \{ *, \epsilon \}$. The reasoning is analogous to that for $FIRST(E')$.

4. $\text{FOLLOW}(E) = \text{FOLLOW}(E') = \{ \}, \$$. Since E is the start symbol, $\text{FOLLOW}(E)$ must contain $\$$. The production body (E) explains why the right parenthesis is in $\text{FOLLOW}(E)$. For E' , note that this nonterminal appears only at the ends of bodies of E -productions. Thus, $\text{FOLLOW}(E')$ must be the same as $\text{FOLLOW}(E)$.
5. $\text{FOLLOW}(T) = \text{FOLLOW}(T') = \{ +,), \$ \}$. Notice that T appears in bodies only followed by E' . Thus, everything except ϵ that is in $\text{FIRST}(E')$ must be in $\text{FOLLOW}(T)$; that explains the symbol $+$. However, since $\text{FIRST}(E')$ contains ϵ (i.e., $E' \rightarrow \epsilon$), and E' is the entire string following T in the bodies of the E -productions, everything in $\text{FOLLOW}(E)$ must also be in $\text{FOLLOW}(T)$. That explains the symbols $\$$ and the right parenthesis. As for T' , since it appears only at the ends of the T -productions, it must be that $\text{FOLLOW}(T') = \text{FOLLOW}(T)$.
6. $\text{FOLLOW}(F) = \{ +, *,), \$ \}$. The reasoning is analogous to that for T in point (5).

LL(1) Grammars:

Predictive parsers, that is, recursive-descent parsers needing no backtracking, can be constructed for a class of grammars called LL(1). The first "L" in LL(1) stands for scanning the input from left to right, the second "L" for producing a leftmost derivation, and the "1" for using one input symbol of lookahead at each step to make parsing action decisions.

The class of LL(1) grammars is rich enough to cover most programming constructs, although care is needed in writing a suitable grammar for the source language. For example, no left-recursive or ambiguous grammar can be LL(1).

A grammar G is LL(1) if and only if whenever $A \rightarrow \alpha \mid \beta$ are two distinct productions of G , the following conditions hold:

1. For no terminal a do both α and β derive strings beginning with a .
2. At most one of α and β can derive the empty string.
3. If $\beta \rightarrow \epsilon$, then α does not derive any string beginning with a terminal in $\text{FOLLOW}(A)$. Likewise, if $\alpha \rightarrow \epsilon$, then β does not derive any string beginning with a terminal in $\text{FOLLOW}(A)$.

The first two conditions are equivalent to the statement that $FIRST(\alpha)$ and $FIRST(\beta)$ are disjoint sets. The third condition is equivalent to stating that if ϵ is in $FIRST(\beta)$, then $FIRST(\alpha)$ and $FOLLOW(A)$ are disjoint sets, and likewise if ϵ is in $FIRST(\alpha)$.

The next algorithm collects the information from $FIRST$ and $FOLLOW$ sets into a predictive parsing table $M[A, a]$, a two-dimensional array, where A is a nonterminal, and a is a terminal or the symbol $\$,$ the input end-marker. The algorithm is based on the following idea: the production $A \rightarrow \alpha$ is chosen if the next input symbol a is in $FIRST(\alpha)$. The only complication occurs when $\alpha = \epsilon$ or, more generally, $\alpha \rightarrow \epsilon$. In this case, we should again choose $A \rightarrow \alpha$, if the current input symbol is in $FOLLOW(A)$, or if the $\$$ on the input has been reached and $\$$ is in $FOLLOW(A)$.

Construction of a predictive parsing table:

INPUT: Grammar G .

OUTPUT: Parsing table M .

METHOD: For each production $A \rightarrow \alpha$ of the grammar, do the following:

1. For each terminal a in $FIRST(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$.
2. If ϵ is in $FIRST(\alpha)$, then for each terminal b in $FOLLOW(A)$, add $A \rightarrow \alpha$ to $M[A, b]$. If ϵ is in $FIRST(\alpha)$ and $\$$ is in $FOLLOW(A)$, add $A \rightarrow \alpha$ to $M[A, \$]$ as well.

If, after performing the above, there is no production at all in $M[A, a]$, then set $M[A, a]$ to error (which we normally represent by an empty entry in the table).

Consider the following grammar

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \epsilon \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \epsilon \\ F &\rightarrow (E) \mid id \end{aligned}$$

The predictive parsing table is shown below:

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Consider production $E \rightarrow TE'$.

Since $FIRST(TE') = FIRST(T) = \{ (, id \}$

This production is added to $M[E, (]$ and $M[E, id]$. Production $E' \rightarrow +TE'$ is added to $M[E', +]$ since $FIRST(+TE') = \{ + \}$. Since $FOLLOW(E) = \{), \$ \}$, production $E' \rightarrow \epsilon$ is added to $M[E',)]$ and $M[E', \$]$.

The grammar of dangling-else problem is

$$S \rightarrow i E t S S' \mid a$$

$$S' \rightarrow e S \mid \epsilon$$

$$E \rightarrow b$$

$FIRST(S) = \{ i, a \}$, $FIRST(S') = \{ e, \epsilon \}$, $FIRST(E) = \{ b \}$

$FOLLOW(E) = \{ t \}$, $FOLLOW(S) = \{ e, \$ \}$, $FOLLOW(S') = \{ e, \$ \}$

The parsing table for this grammar is shown below:

NON - TERMINAL	INPUT SYMBOL					
	a	b	c	i	t	$\$$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow \epsilon$ $S' \rightarrow eS$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				

The entry for $M[S', e]$ contains both $S' \rightarrow eS$ and $S' \rightarrow \epsilon$.

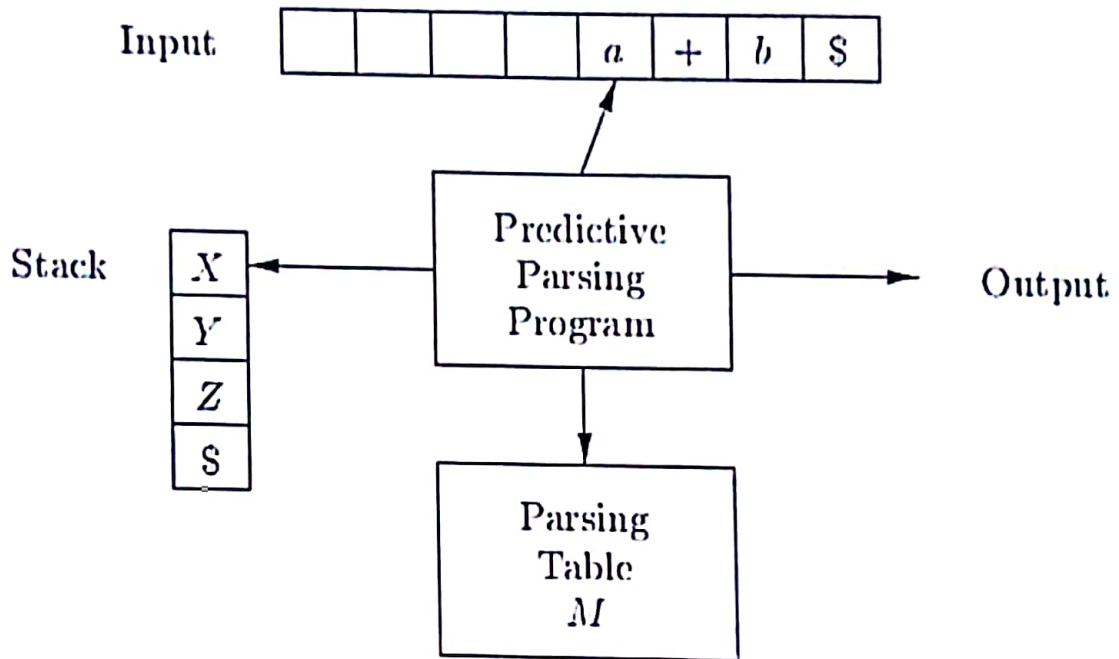
The grammar is ambiguous and the ambiguity is manifested by a choice in what production to use when an e (else) is seen. We can resolve this ambiguity by choosing $S' \rightarrow eS$. This choice corresponds to associating an else with the closest previous then. Note that the choice $S' \rightarrow \epsilon$ would prevent e from ever being put on the stack or removed from the input, and is surely wrong.

Nonrecursive Predictive parsing:

A nonrecursive predictive parser can be built by maintaining a stack explicitly, rather than implicitly via recursive calls. The parser mimics a leftmost derivation. If w is the input that has been matched so far, then the stack holds a sequence of grammar symbols α such that

$$S \rightarrow_{lm} w\alpha$$

Model of a table-driven predictive parser is shown below:



The table-driven parser has an input buffer, a stack containing a sequence of grammar symbols, a parsing table constructed by Algorithm (for **Construction of a predictive parsing table**), and an output stream. The input buffer contains the string to be parsed, followed by the endmarker \$. We reuse the symbol \$ to mark the bottom of the stack, which initially contains the start symbol of the grammar on top of \$.

The parser is controlled by a program that considers X , the symbol on top of the stack, and a , the current input symbol. If X is a nonterminal, the parser chooses an X -production by consulting entry $M[X, a]$ of the parsing table M . Otherwise, it checks for a match between the terminal X and current input symbol a .

The behavior of the parser can be described in terms of its configurations, which give the stack contents and the remaining input. The next algorithm describes how configurations are manipulated.

Table-driven predictive parsing:

INPUT: A string w and a parsing table M for grammar G .

OUTPUT: If w is in $L(G)$, a leftmost derivation of w ; otherwise, an error indication.

METHOD: Initially, the parser is in a configuration with $w\$$ in the input buffer and the start symbol S of G on top of the stack, above $\$$. The program shown below uses the predictive parsing table M to produce a predictive parse for the input.

```
let  $a$  be the first symbol of  $w$ ;  
let  $X$  be the top stack symbol;  
while (  $X \neq \$$  ) { /* stack is not empty */  
    if (  $X = a$  ) pop the stack and let  $a$  be the next symbol of  $w$ ;  
    else if (  $X$  is a terminal ) error();  
    else if (  $M[X, a]$  is an error entry ) error();  
    else if (  $M[X, a] = X \rightarrow Y_1 Y_2 \cdots Y_k$  ) {  
        output the production  $X \rightarrow Y_1 Y_2 \cdots Y_k$ ;  
        pop the stack;  
        push  $Y_k, Y_{k-1}, \dots, Y_1$  onto the stack, with  $Y_1$  on top;  
    }  
    let  $X$  be the top stack symbol;  
}
```

Consider the following grammar

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \varepsilon \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \varepsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

The predictive parsing table is shown below:

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

On input $id + id * id$, the nonrecursive predictive parser of Algorithm makes the sequence of moves shown below. These moves correspond to a leftmost derivation.

MATCHED	STACK	INPUT	ACTION
	$E\$$	$id + id * id\$$	
	$TE'\$$	$id + id * id\$$	output $E \rightarrow TE'$
	$FT'E'\$$	$id + id * id\$$	output $T \rightarrow FT'$
	$id T'E'\$$	$id + id * id\$$	output $F \rightarrow id$
id	$T'E'\$$	$+ id * id\$$	match id
id	$E'\$$	$+ id * id\$$	output $T' \rightarrow \epsilon$
id	$+ TE'\$$	$+ id * id\$$	output $E' \rightarrow + TE'$
$id +$	$TE'\$$	$id * id\$$	match $+$
$id +$	$FT'E'\$$	$id * id\$$	output $T \rightarrow FT'$
$id +$	$id T'E'\$$	$id * id\$$	output $F \rightarrow id$
$id + id$	$T'E'\$$	$* id\$$	match id
$id + id$	$* FT'E'\$$	$* id\$$	output $T' \rightarrow * FT'$
$id + id *$	$FT'E'\$$	$id\$$	match $*$
$id + id *$	$id T'E'\$$	$id\$$	output $F \rightarrow id$
$id + id * id$	$T'E'\$$	$\$$	match id
$id + id * id$	$E'\$$	$\$$	output $T' \rightarrow \epsilon$
$id + id * id$	$\$$	$\$$	output $E' \rightarrow \epsilon$